✅ What is a CTE?

CTE (Common Table Expression) is:

A temporary named result set that exists only during the execution of one SQL query.

It works like:

- A temporary table
- Or a temporary view
- But only inside one query

🎯 Why Use CTE?

CTEs help to:

✓ Make complex queries readable
✓ Break big queries into smaller parts
✓ Reuse subqueries
✓ Improve maintainability
✓ Write recursive queries

🧠 Simple Idea

Instead of writing one long confusing query 😵🌀

You write:

1. First define logic
2. Give it a name
3. Use it like a table

📌 Basic Syntax of CTE

```
WITH cte_name AS (
  SELECT column1, column2
  FROM table_name
  WHERE condition
)
SELECT *
FROM cte_name;
```

🔎 **Parts Explained**

1️⃣ **WITH Clause**

Starts the CTE definition.

2️⃣ **CTE Query**

Defines the temporary result set.

3️⃣ **Main Query**

Uses the CTE like a table.

🔥 **Example 1: Simple Filtering**

Without CTE:

```
SELECT id, name, city, purchase_amount
FROM messy_indian_dataset
WHERE city = 'Mumbai'
AND purchase_amount > 1000
AND gender = 'Male';
```

With CTE:

```
WITH MumbaiHighSpenders AS (
  SELECT id, name, city, purchase_amount
  FROM messy_indian_dataset
  WHERE city = 'Mumbai'
  AND purchase_amount > 1000
  AND gender = 'Male'
)
SELECT *
FROM MumbaiHighSpenders;
```

✅ **Why Better?**

- Cleaner
- More readable
- Logic separated

🔥 **Example 2: Complex Query Simplified**

```
WITH ComplexQuery AS (
  SELECT c.customer_id, c.name,
       SUM(o.order_total) AS total_spent
  FROM customers c
  JOIN orders o
    ON c.customer_id = o.customer_id
  WHERE c.city = 'New York'
   AND o.order_date >= '2023-01-01'
  GROUP BY c.customer_id, c.name
  HAVING SUM(o.order_total) > 5000
)
SELECT *
FROM ComplexQuery;
```

What Happens?

1. CTE calculates total spending
2. Filters customers spending > 5000
3. Main query just selects result

👉 Makes heavy logic easier to understand.

🔥 **Example 3: Reusing Subquery**

```
WITH TopCustomers AS (
  SELECT customer_id,
       SUM(order_total) AS total_spent
  FROM orders
  GROUP BY customer_id
  HAVING SUM(order_total) > 10000
)
SELECT c.name, tc.total_spent
FROM customers c
JOIN TopCustomers tc
```

```
  ON c.customer_id = tc.customer_id;
```

Why CTE Helps?

- Avoids repeating SUM logic
- Cleaner join
- Easy to maintain

🔥 Example 4: Pre-Aggregation (Performance Use)

```
WITH PreAggregated AS (
  SELECT city,
      COUNT(*) AS num_purchases,
      SUM(purchase_amount) AS total_purchase,
      AVG(purchase_amount) AS avg_purchase,
      MAX(purchase_amount) AS max_purchase
  FROM messy_indian_dataset
  GROUP BY city
)
SELECT *
FROM PreAggregated
WHERE total_purchase > 100;
```

Why Useful?

- Aggregation done once
- Reduces repeated calculations
- Makes main query simple

🚀 Advantages of CTE

1️⃣ Improves Readability

Instead of messy nested queries:

```
SELECT * FROM (
  SELECT ...
) AS temp;
```

Use clean CTE.

2️⃣ Code Reusability

You can reference CTE multiple times in same query.

3️⃣ Better Debugging

You can:

- Run only CTE part
- Test logic step by step

4️⃣ Supports Recursive Queries 🔥

CTEs allow recursion.

Used for:

- Organizational hierarchy
- Bill of materials
- Tree structures

Example:

```
WITH RECURSIVE EmployeeHierarchy AS (
  SELECT employee_id, manager_id
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  SELECT e.employee_id, e.manager_id
  FROM employees e
  JOIN EmployeeHierarchy eh
  ON e.manager_id = eh.employee_id
)
SELECT * FROM EmployeeHierarchy;
```

📊 CTE vs Subquery

CTE	Subquery
More readable	Can become messy
Named result	No name
Can reuse	Hard to reuse
Supports recursion	Does not

⚠️ Important Notes

- CTE exists only for that query
- Cannot be indexed
- Not stored permanently
- Sometimes materialized (depends on DB engine)

🧠 When to Use CTE?

Use CTE when:

✓ Query is complex
✓ Multiple joins
✓ Multiple aggregations
✓ Need recursion
✓ Need better readability

❌ When Not Necessary?

For very simple queries, CTE is not required.

🎯 Quick Summary

CTE = Temporary Named Query

Syntax:

WITH name AS (query)

SELECT * FROM name;

Benefits:

✓ Cleaner queries
✓ Easier maintenance
✓ Reusable logic
✓ Supports recursion